Listing 5.33 Updating the document if the field doesn't exist

```
POST movies/_update/11
{
  "doc": {
    "runtime_in_minutes":110
  },
  "doc_as_upsert":true
}
```

This time, if we don't have a document with ID 11 in our store, it's not a problem. The engine does not throw an error; instead, it creates a new document with ID 11 and fields extracted from the `doc` object (`runtime_in_minutes`, in this case).

5.4.7 Updating with a query

Sometimes we wish to update a set of documents matching a search criterion—for example, updating all movies with a rating greater than 4 stars to indicate that they are popular. We can run a query to search for such a set of documents and apply updates to them by using the `_update_by_query` endpoint. For example, let's update all movies with an actor name matching `Al Pacino` to say `Oscar Winner Al Pacino`, instead.

Listing 5.34 Updating a document using a `query` method

```
POST movies/_update_by_query
{
  "query": {
    "match": {
      "actors": "Al Pacino"
    }
  },
  "script": {
    "source": """
    ctx._source.actors.add('Oscar Winner Al Pacino');
    ctx._source.actors.remove(ctx._source.actors.indexOf('Al Pacino'))
    """,
    "lang": "painless"
  }
}
```

Searches for documents and updates the set

Search query: searches for all movies with Al Pacino

Applies the following script logic to the matching documents

In this listing, the `match` query executes first (we discuss `match` queries in chapter 10, but think of it as a type of query where we can fetch documents matching a given criterion) to fetch all movies with actor Al Pacino. When the `match` query returns its results, a script executes to change `Al Pacino` to `Oscar Winner Al Pacino`. Because we need to remove the old name, we also invoke a `remove` operation in the script.

`_update_by_query` is a handy mechanism to update many documents based on a criterion. However, there is a lot going on behind the scenes when we use this method; see the following sidebar to learn more.